COMP3230 · Principles of Operating Systems

Tutorial 2

Process

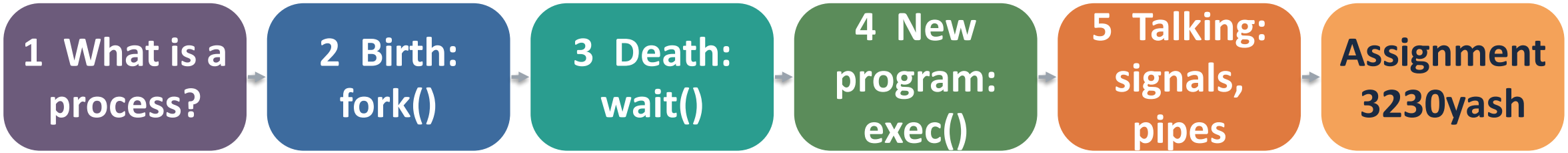
```
systemd(1)
└─ sshd
   └─ bash
      └─
./a.out
```

`fork · exec · wait · signals · pipes`

Teaching Assistant · Miao Yu · miaoyu.hku@connect.hku.hk

Today's goals — and the road we will take

- **Explain** what a process is: address space, registers, PCB, states
- **Create** processes with fork(); PIDs, copy-on-write, random order
- **Control termination:** exit(), wait() family, zombies, /proc statistics
- **Run other programs** with exec() — why a shell is fork–exec–wait
- **Communicate** with signals and pipes, e.g. ls | grep



The order of the parts is the order in which a shell uses the OS.

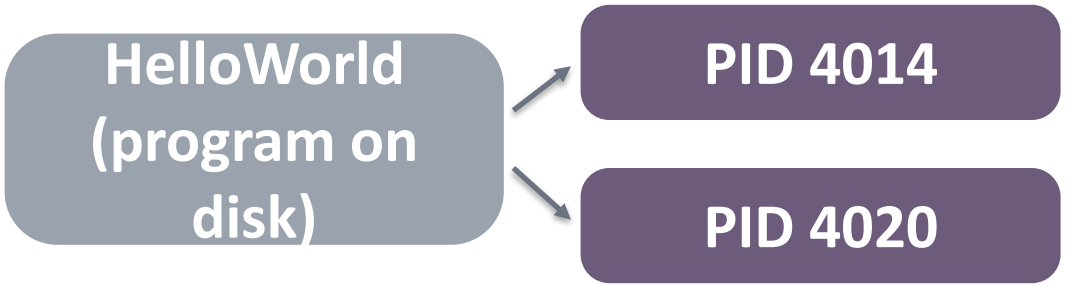
PART 1

The process abstraction — what the OS must remember

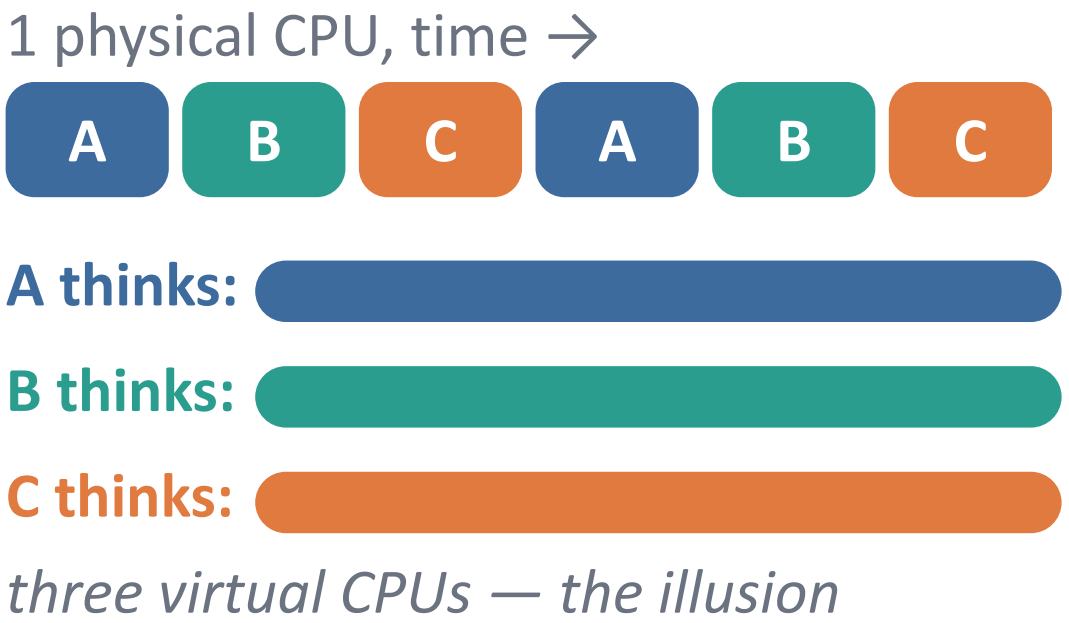
- What is a process, and how is it different from a program? Why do we need it?
- What exactly must the OS remember about a running program?
- Which states does a process go through?
- Which data structures track all this — and how can I see them from the shell?

Program vs. process — and why processes exist

- **Program:** lifeless bytes on disk
- **Process:** a program in execution — an instance of it
- One program can be many processes
- **The crux:** few CPUs, many programs → time sharing
- **Mechanism** (how): context switch
- **Policy** (which): scheduler



run it twice → two processes



three virtual CPUs — the illusion

What makes up a process: its machine state

Memory

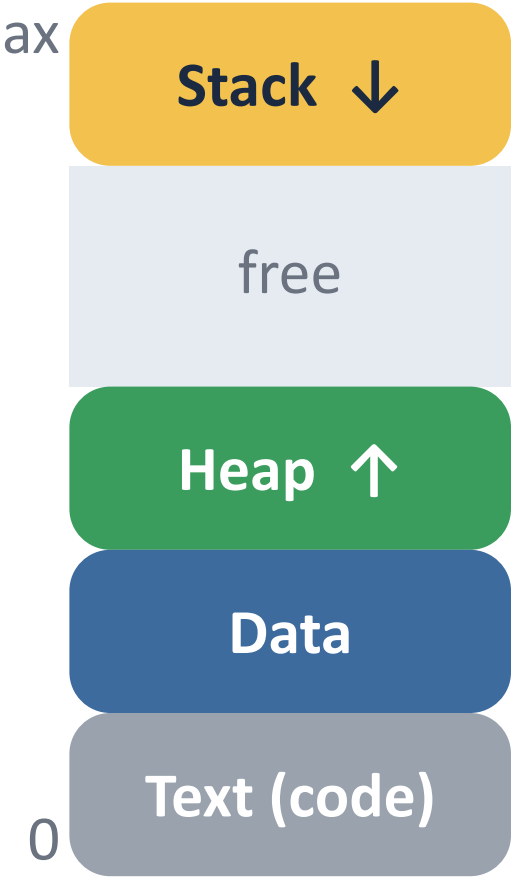
address space:
code, data, heap, stack

Execution state

registers (PC, SP, …)
+ current process state

Resources

open files (0, 1, 2),
cwd, credentials

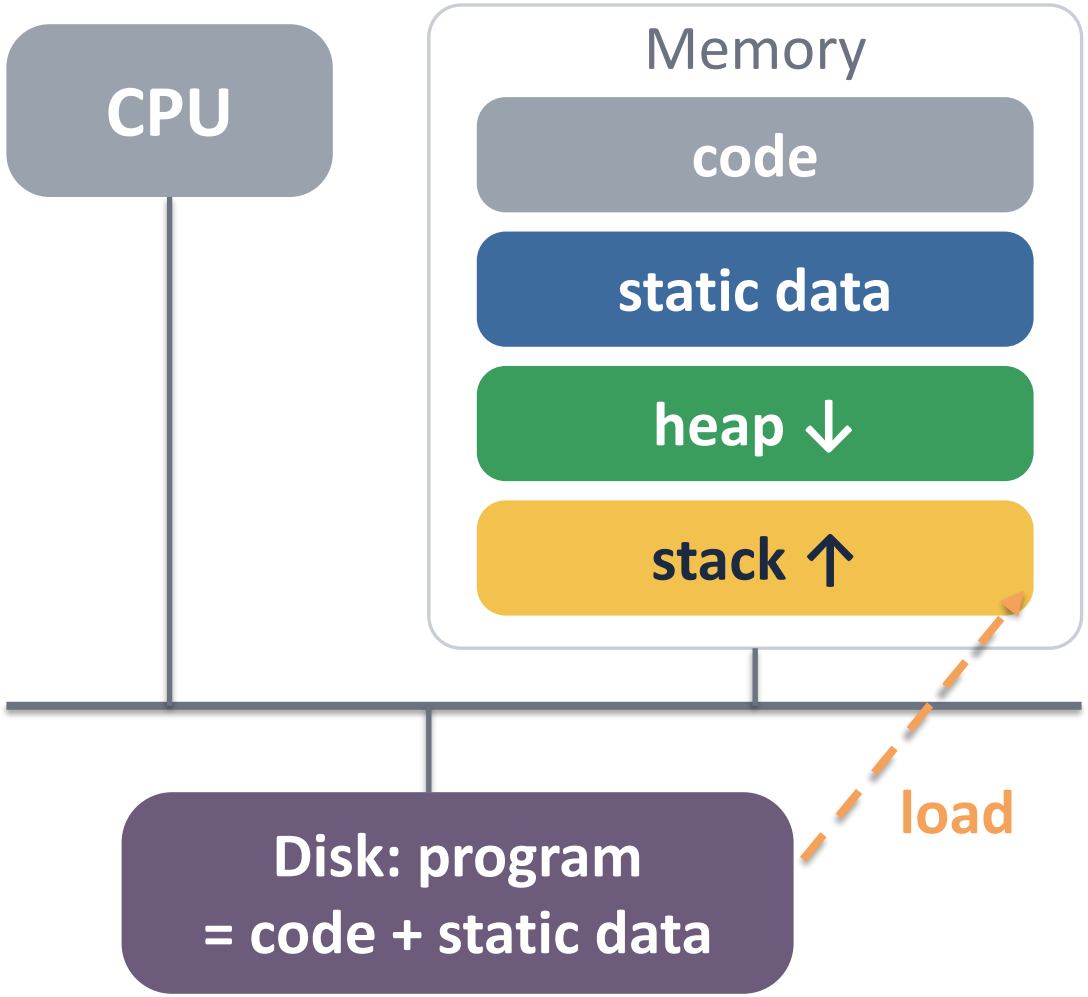


address-space.c

```
int g = 100;                  // data
int main(void) {              // text
  int local = 1;              // stack
  int *p = malloc(16);        // p: stack
                              // 16 B: heap
  return 0;
}
```

The address space is what fork() copies and exec() re-initializes.

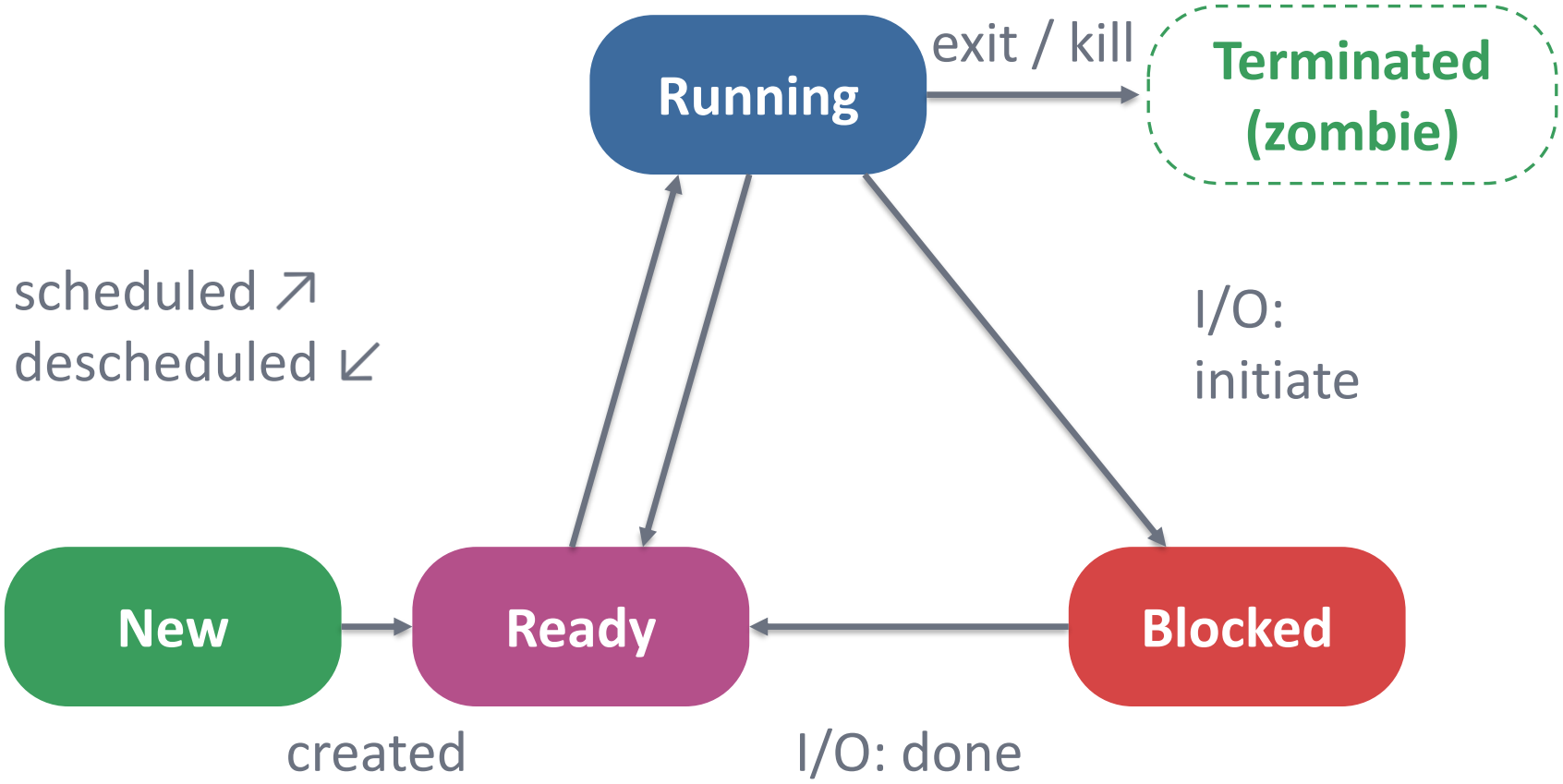
From program to process: loading



- Load code + static data (eagerly, or lazily on demand)
- Set up the stack — fill in argc, argv
- Set up the heap
- Open file descriptors 0, 1, 2
- Fill in the PCB: PID, PPID, PC, SP → Ready queue
- Jump to main()

Windows: CreateProcess() · Unix: fork()+exec()

Process states: the life cycle

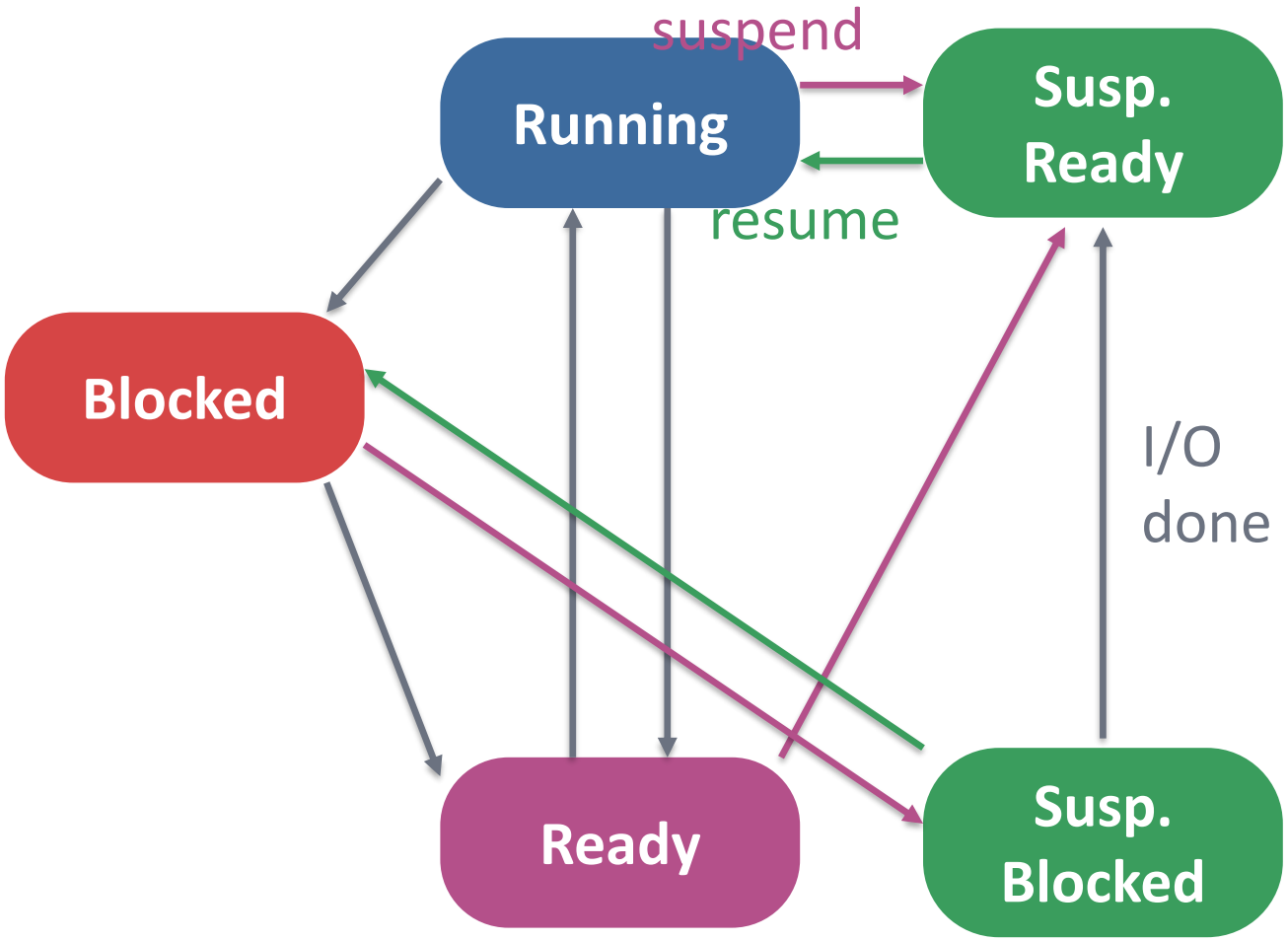


t	P0	P1
1	Run	Ready
2	Run	Ready
3	I/O →	Ready
4	Blk	Run
5	Blk	Run
6	Blk	Run
7	Ready	Run
8	Ready	done
9	Run	–
10	Run	–

Trace: P0 blocks → P1 runs. I/O done at t7, yet P1 keeps running.

Suspension: an external pause

- Suspend = remove from scheduling; another process must resume it
- Why: user (Ctrl-Z … fg), the parent, or the OS freeing memory
- **Blocked** = internal cause (it asked for I/O)
- **Suspended** = external cause



magenta = suspend **green** = resume

The Process Control Block (PCB)

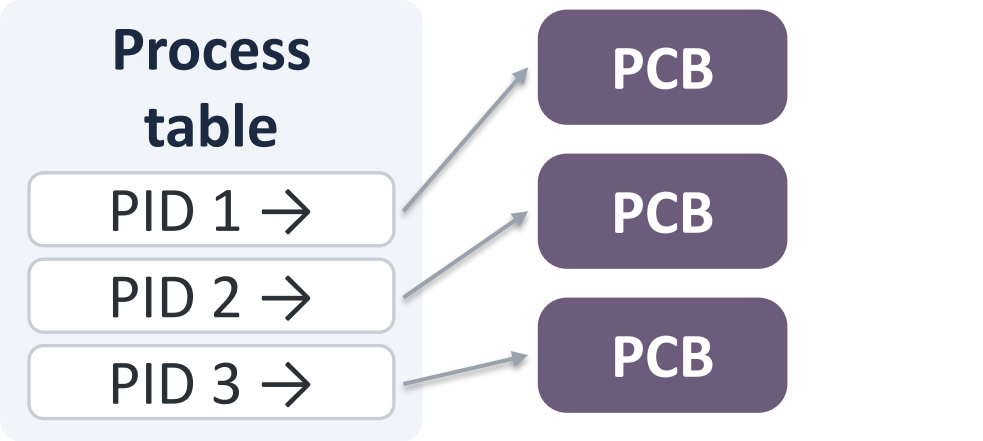
- PID, state, program counter
- Register context (saved on a switch)
- Scheduling info, credentials, accounting
- Memory map, open files, cwd
- Parent and children pointers
- Signal handlers (Part 5)
- **Linux:** `task_struct` ≈ 800 lines

`xv6-proc.h` (OSTEP Fig. 4.5)

```
enum proc_state { UNUSED, EMBRYO,
  SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
struct proc {
  char *mem;  uint sz;          // memory
  enum proc_state state;        // state
  int pid;                      // PID
  struct proc *parent;          // parent
  struct file *ofile[NOFILE]; // files
  struct inode *cwd;            // cwd
  struct context context;       // regs
  struct trapframe *tf;
};
```

EMBRYO = New, ZOMBIE = Terminated, SLEEPING = Blocked

Process table, process lists — and how to see them



Current (per core) PCB

Ready list PCB → PCB → PCB

Blocked list PCB → PCB

Reaped → removed from the table, resources freed

```
$ ps
    PID TTY          TIME CMD
3120691 pts/1    00:00:00 bash
3123097 pts/1    00:00:00 ps
$ pstree -sp 3120691
systemd(1)───sshd(205)───sshd(3120553)
  ───sshd(3120626)───bash(3120691)
  ───pstree(3123102)
```

- `ps -e -f` all processes, full format
- `ps -o pid,ppid,stat,cmd`
- `pstree -sp <pid>` · `top` · `kill`
- PID 1 = systemd (init): root of the tree

Part 1 recap → operations on processes

- Process = running program, described by a PCB
- Time sharing: mechanism vs. policy
- New → Ready ⇄ Running → Blocked; → Terminated (zombie)
- Process table + Current / Ready / Blocked lists

Operation	Unix call
create	`fork()+exec()`
destroy	`exit(), kill()`
wait / status	`wait*(), /proc`
run program	`exec*()`
suspend	`SIGSTOP/CONT`
IPC	`signals, pipes`

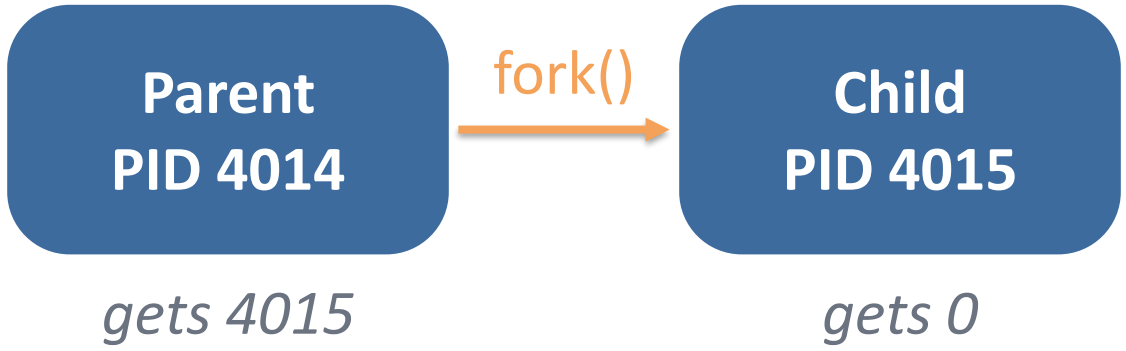
Next: the system calls behind these operations — starting with fork().

fork(): one call, two returns

```
pid_t fork(void);     // <unistd.h>
```

- Duplicates the caller: code, data, heap, stack, open fds
- The child continues right after fork() — not at main()
- **Called once, returns twice**
- What differs: PID, PPID — and the return value

fork() returns	meaning
< 0	failed — no child created
== 0	I am the child
> 0	I am the parent; value = child's PID



Code: fork-example1.c — who am I?

fork-example1.c

```
pid_t pid = fork();
if (pid < 0) {                    // (1) error
    fprintf(stderr, "fork() Failed");
    exit(-1);
} else if (pid == 0) {            // (2) child
    printf("Child Process.\n");
    printf("pid  = %d\n", getpid());
    printf("ppid = %d\n", getppid());
} else {                          // (3) parent
    printf("Parent Process.\n");
    printf("pid  = %d\n", getpid());
    printf("ppid = %d\n", getppid());
}
return 0;
```

output (two runs)

```
$ ./fork-example1              # run 1
Parent Process.
pid  = 4014
ppid = 2514
Child Process.
pid  = 4015
ppid = 1          ← !!
$ ./fork-example1              # run 3
Child Process.
pid  = 4263
ppid = 4262       ← parent alive
Parent Process.   ...
```

- `getpid()` = my PID · `getppid()` = my parent's PID
- Expected: child.ppid == parent.pid … but run 1 says 1?

Same file, two processes, two paths through the if.

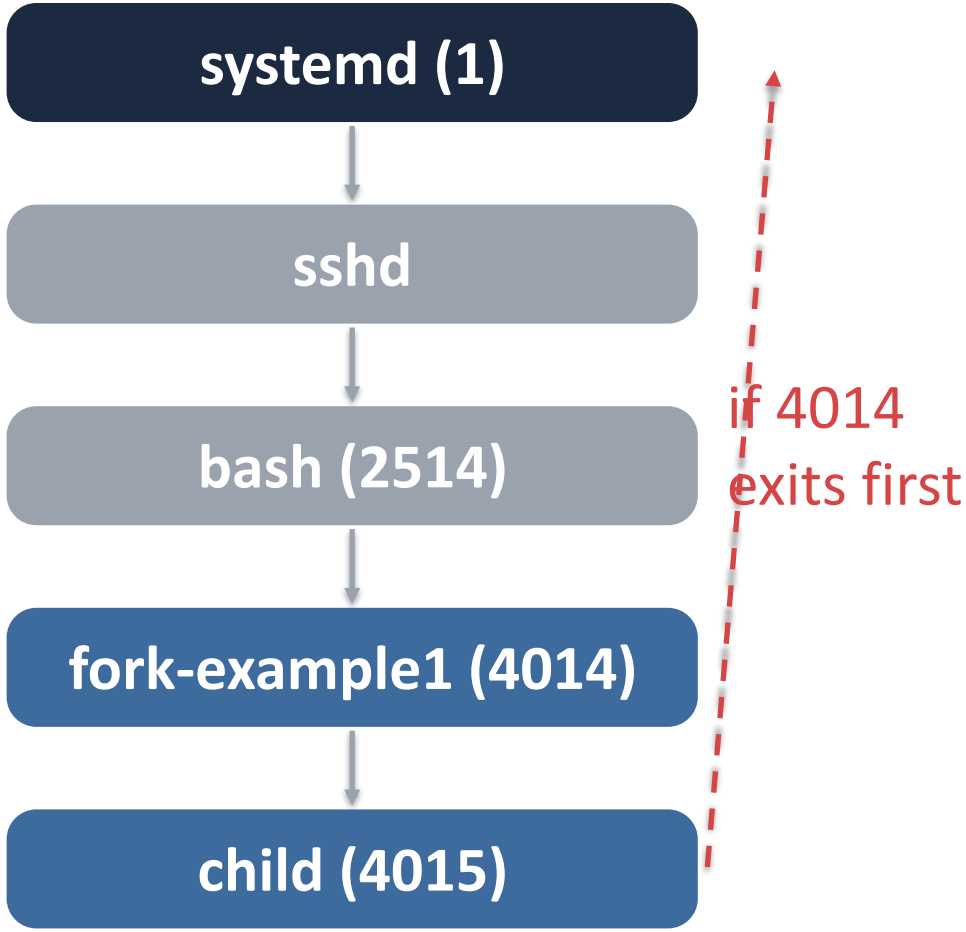
Two lessons: non-determinism and orphans

Lesson 1 — the scheduler decides

Two runnable processes → never assume who runs first.
sleep() in the parent? No guarantee. wait()? Yes → Part 3.

Lesson 2 — orphans

The parent exited first → the child is re-parented to PID 1 (systemd / init), which will reap it later.

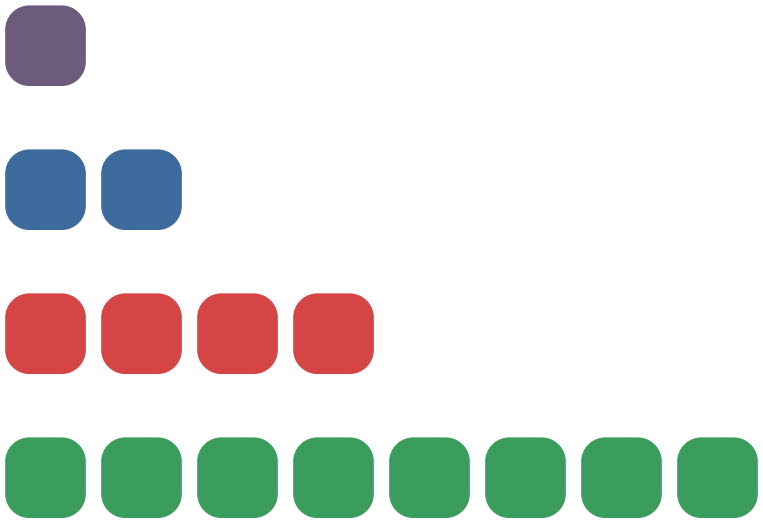


Every process but PID 1 has a parent → a tree.

How many processes? Multiple fork()s

fork-example2.c

```
fork(); fork(); fork();
printf("Hello World!\n"); // 8 times
```



1 · start

2 · 1st fork()

4 · 2nd fork()

8 · 3rd fork()

n forks → 2^n processes; each prints once

Q1

```
int c = 100;
fork();
if (c == 100) printf("X");
```

→ **2 × X**

c is in the data segment → copied

Q2

```
pid = fork();
if (pid == 0) printf("X");
printf("Y");
```

→ **1 × X, 2 × Y**

only the child has pid == 0; both print Y

After fork(), every variable exists twice.

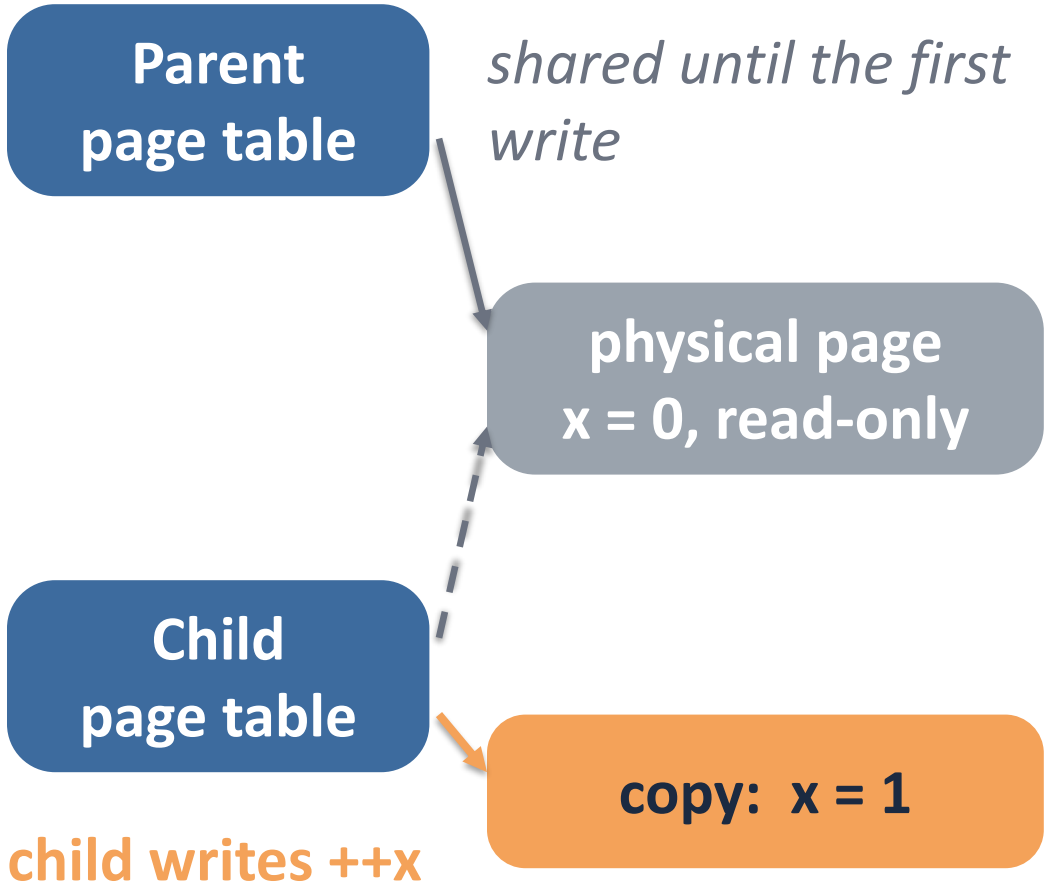
Copy-on-write: how the copy is made cheap

fork-example3-COW.c

```
int x = 0;
int rc = fork();
if (rc == 0)    // child
  printf("child:  &x=%p x=%d\n", &x, ++x);
else            // parent
  printf("parent: &x=%p x=%d\n", &x, --x);
```

```
child:  &x=0x7ffd3c1a  x=1
parent: &x=0x7ffd3c1a  x=-1
← same address, different values
```

- Pages are shared, marked read-only
- First write → page fault → copy that page
- fork() stays cheap → fork-then-exec is fine



Debugging fork() with gdb / VS Code

- Compile with `-g`
- Default: follow the **parent**, detach the child
- `set follow-fork-mode child`
- `set detach-on-fork off`: keep both
- `b r n s finish p c bt`
- Also: `printf(getpid())`

`.vscode/launch.json`

```
"setupCommands": [
  { "text": "-enable-pretty-printing" },
  { "text": "-gdb-set detach-on-fork off" }
]
```

gdb-fork-debug.md in the code folder has the full command list.

Next → random order, orphans, zombies … Part 3 adds order.

PART 3

Termination & synchronization: exit(), the wait() family, /proc

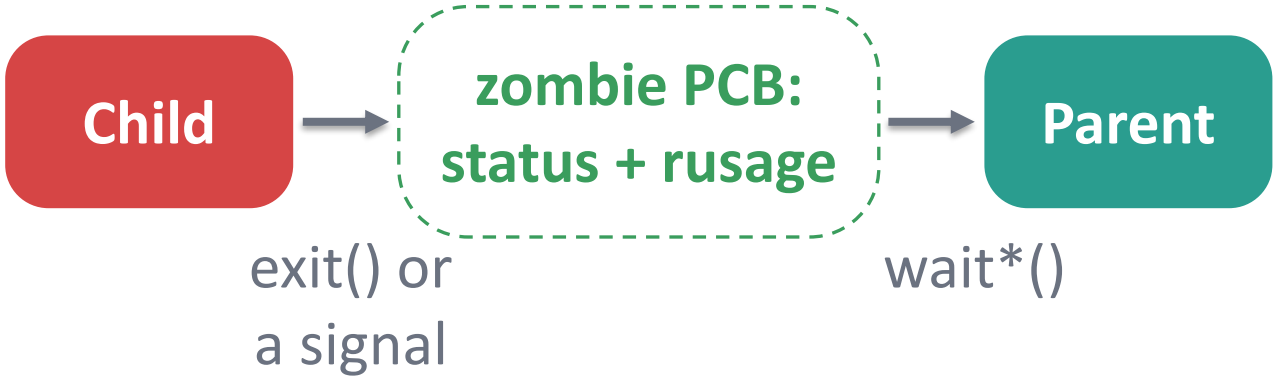
Where we are

- fork() clones the caller (copy-on-write)
- Returns child PID / 0 / −1; child continues after fork()
- Two processes → the scheduler decides the order
- n forks → 2^n processes; every variable exists twice

Questions for this part

- How does a process end, and what is left behind?
- Zombies and orphans — why keep a dead process?
- Waiting: any child, a specific child, or without blocking?
- Exit status, killing signal, CPU time, live stats in /proc?

How a process dies — zombies and orphans



- **Voluntary:** exit(status) or return from main(); 0 = success; low 8 bits kept
- **Involuntary:** a signal — kill, Ctrl-C, SIGSEGV, SIGFPE …
- Kernel keeps a minimal PCB and sends SIGCHLD; freed after wait*()

Zombie — dead but not yet waited for. Keeps exit status + usage; costs a PID.

Orphan — alive, parent dead → adopted by PID 1, reaped later.

```
$ ps -o pid,ppid,stat,cmd
 5120 2514 S   ./make-zombie
 5121 5120 Z   [make-zombie] <defunct>
```

wait() and waitpid(): three possible outcomes

```
#include <sys/wait.h>
pid_t wait(int *status);                                  // any child
pid_t waitpid(pid_t pid, int *status, int options);       // specific child
```

Children still running

→ block (parent: Blocked)

A child is already a zombie

→ return its status now

No children at all

→ −1, errno = ECHILD

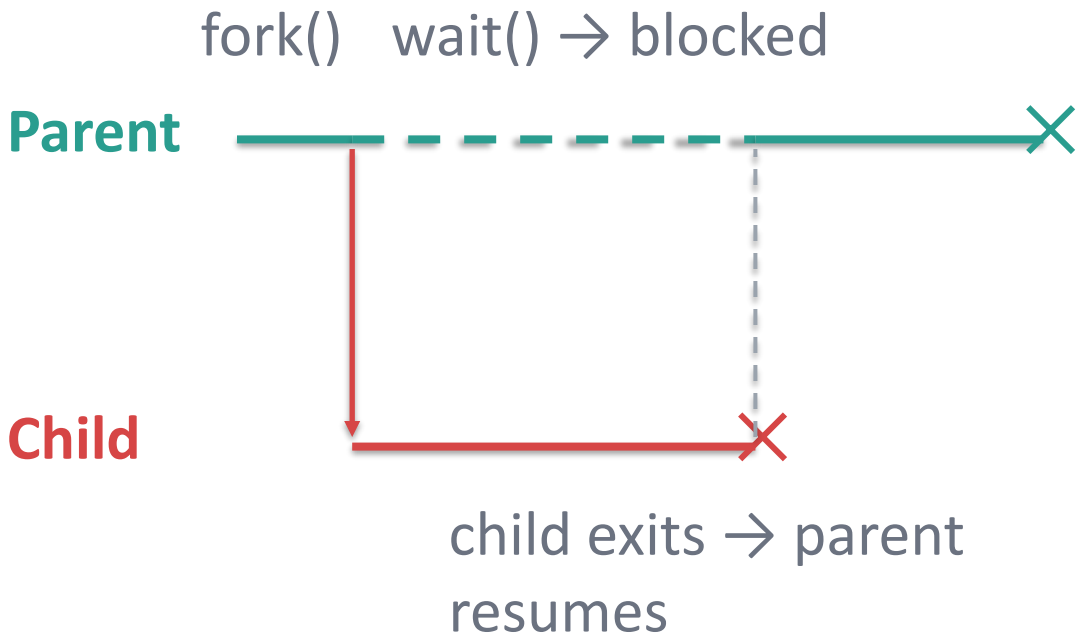
- When a child ends, the kernel sends SIGCHLD to the parent (asynchronous)
- Return value: the child's PID (or −1); status may be NULL
- One call collects one child — loop to reap several

Code: fork-example4-wait.c — one line, deterministic

fork-example4-wait.c (diff)

```
} else {                        // parent
    wait(NULL);                 // <-- the only new line
    printf("Parent Process.\n");
    printf("pid  = %d\n", getpid());
    printf("ppid = %d\n", getppid());
}
```

```
$ ./fork-example4-wait      # every run
Child Process.
pid  = 11376
ppid = 11375      ← real parent
Parent Process.
pid  = 11375
```

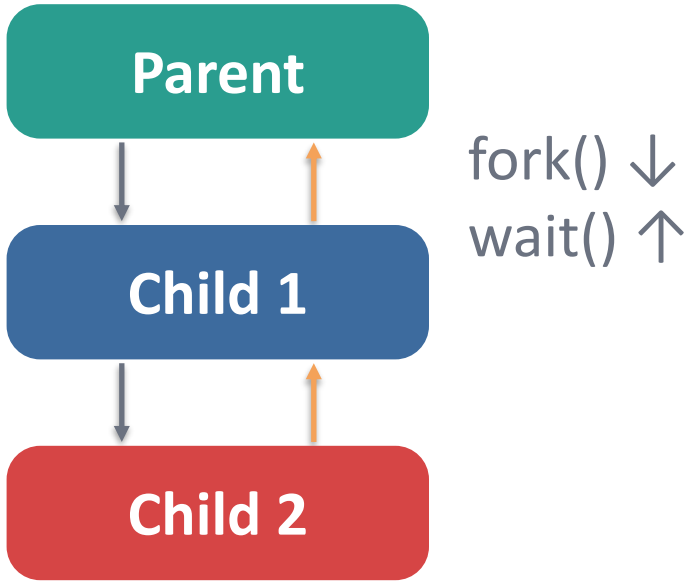


- Child first? fine. Parent first? it blocks in wait() → child still prints first
- The child's ppid is now always the real parent

Code: fork-wait-example1.c — nested fork + wait

fork-wait-example1.c

```
pid1 = fork();
if (pid1 == 0) {                    // Child 1
    pid2 = fork();
    if (pid2 == 0) {                // Child 2
        printf("Child Process 2\n");
    } else {
        wait(NULL);                 // C1 waits for C2
        printf("Child Process 1\n");
    }
} else {
    wait(NULL);                     // P waits for C1
    printf("Parent Process\n");
}
```




```
$ ./fork-wait-example1
Child Process 2
Child Process 1
Parent Process
```

Each process reaps its own children.

Decoding the status word

fork-wait-example2-status.c

```
int status;  pid_t pid = fork();
if (pid == 0) { printf("Child.\n"); exit(3); }
wait(&status);
if (WIFEXITED(status))
    printf("exit code %d\n",
           WEXITSTATUS(status));
else if (WIFSIGNALED(status))
    printf("killed by %s\n",
           strsignal(WTERMSIG(status)));
```



- WIFEXITED → WEXITSTATUS
- WIFSIGNALED → WTERMSIG
- strsignal(sig) → "Interrupt"
- WIFSTOPPED → WSTOPSIG

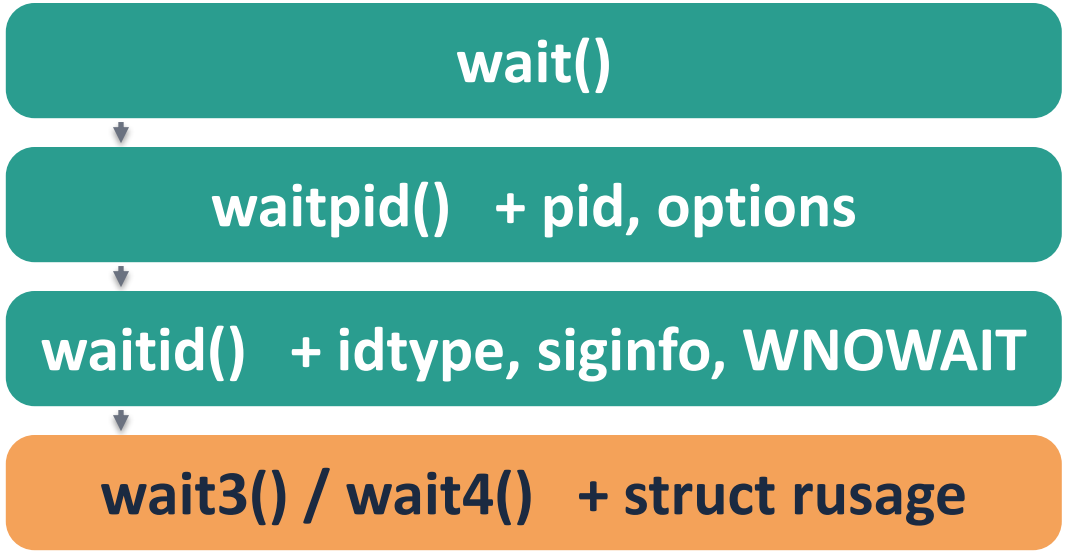
status is encoded — use the macros.

Assignment: loopf: Interrupt = WIFSIGNALED + strsignal(WTERMSIG(st))

The whole wait family

pid	waitpid() waits for
`> 0`	that child
`-1`	any child (= wait)
`0`	any child in my process group
`< -1`	any child in group |pid|

- `WNOHANG`: don't block; 0 if nothing yet
- `WUNTRACED / WCONTINUED`: stopped / continued
- `WNOWAIT` (waitid): report, but keep the zombie



wait4-rusage.c

```
wait4(pid, &st, 0, &ru);          // + rusage
double u = ru.ru_utime.tv_sec
         + ru.ru_utime.tv_usec / 1e6;
waitid(P_PID, pid, &info,   // waitid-nowait.c
       WEXITED | WNOHANG | WNOWAIT);
```

Live statistics: /proc/<pid>/stat

#	field	meaning
3	`state`	R, S, D, T, Z (zombie)
10	`minflt`	minor page faults
12	`majflt`	major page faults
14	`utime`	user time, ticks
15	`stime`	kernel time, ticks
23	`vsize`	virtual memory, bytes
39	`processor`	last CPU

ticks → s: ÷ `sysconf(_SC_CLK_TCK)`

`proc-stat-read.c`

```
snprintf(path, 64, "/proc/%d/stat", pid);
FILE *f = fopen(path, "r");
fgets(buf, sizeof buf, f);  fclose(f);
char *p = strrchr(buf, ')') + 2; // after (comm)
sscanf(p, "%c %*d %*d %*d %*d %*d %*u %lu "
          "%*u %lu %*u %lu %lu", &state,
          &minflt, &majflt, &utime, &stime);
```

- comm may contain spaces: parse after the last ')'
- The file still exists for a zombie (state Z)

Next → so far every child runs the same program → exec()

PART 4

Running a new program: exec() and the fork–exec–wait pattern

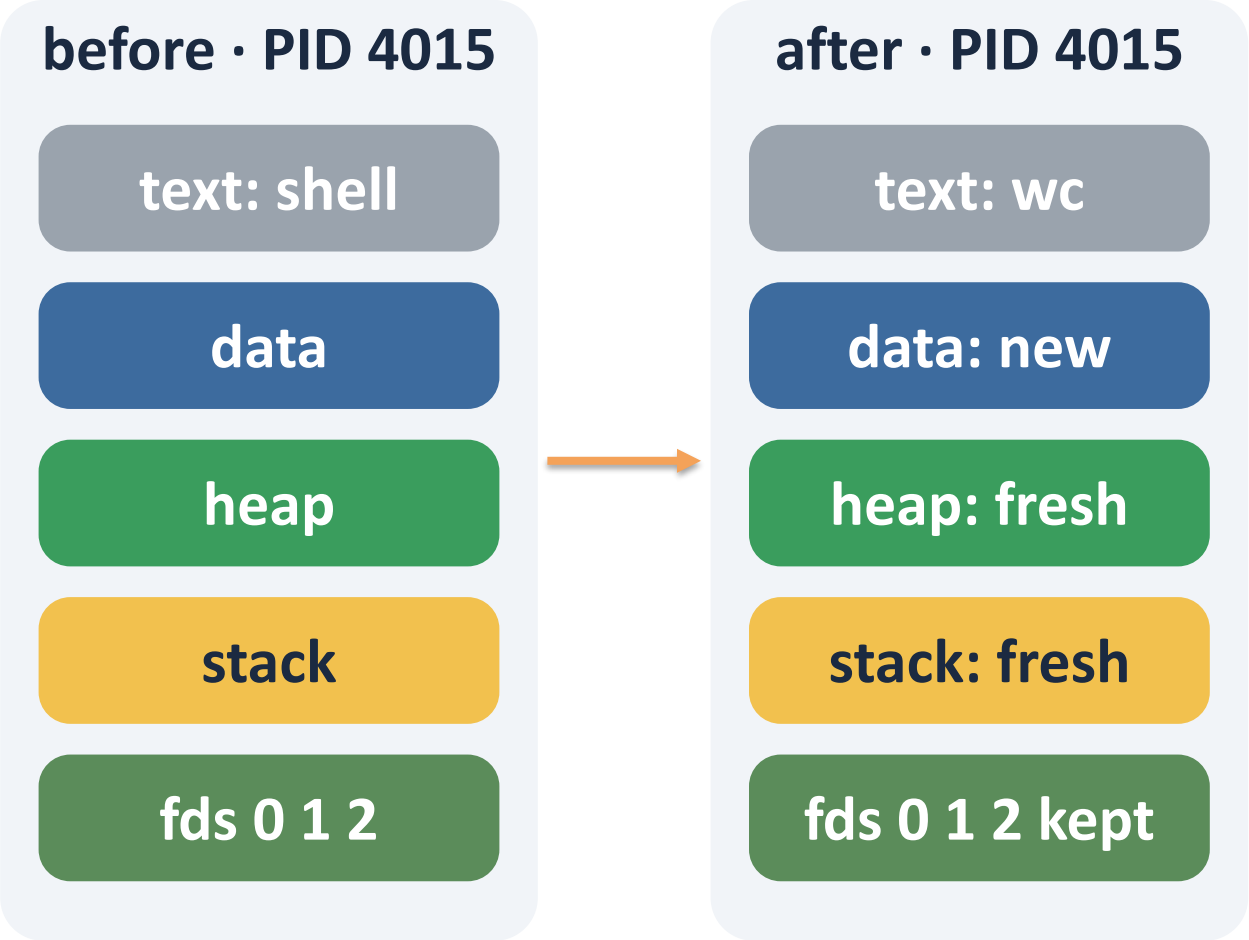
Where we are

- exit()/signal ends a process → zombie PCB + SIGCHLD
- wait / waitpid (WNOHANG) / waitid (WNOWAIT) / wait4 (rusage)
- WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG
- Orphans → PID 1; /proc/<pid>/stat for live statistics

Questions for this part

- How does a process start running a different program?
- Why six (seven) exec functions?
- Why fork() + exec() instead of one CreateProcess()?
- What can a shell do in the gap between them?

exec(): a new program, the same process



`execvp("wc")` ✗ *never returns*

- Loads a new program over the process
- Heap and stack re-initialized; starts at main()
- Same PID, same PPID, same open fds
- Never returns on success; −1 + errno on failure
- Caught handlers → default; ignored stay ignored

The exec family: seven front doors, one system call

letter	meaning
`l`	arguments as a NULL-ended list
`v`	arguments as a vector argv[]
`p`	search $PATH for the file
`e`	pass an explicit envp[]

- argv[0] = program name; array ends with NULL
- p-variants do the $PATH search for you
- −1 on error; no return on success
- execve() is the real system call

`fork-exec-wait-example4.c: ls -l seven ways`

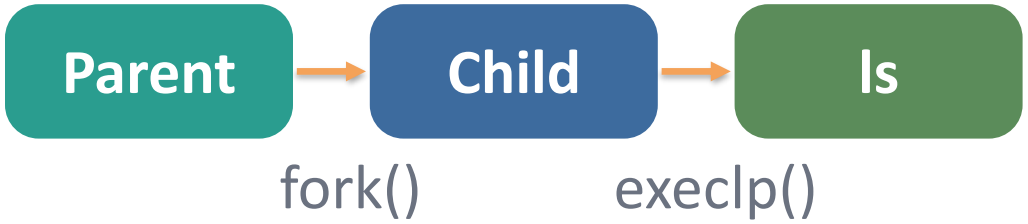
```
char *argv[] = {"ls", "-l", NULL};  char *envp[] = {NULL};
execvp("ls", argv);                          // PATH + vector
execlp("ls", "ls", "-l", NULL);              // PATH + list
execv("/bin/ls", argv);                      // path + vector
execl("/bin/ls", "ls", "-l", NULL);          // path + list
execle("/bin/ls", "ls", "-l", NULL, envp); // + environment
execve("/bin/ls", argv, envp);               // the syscall
execvpe("ls", argv, envp);                   // GNU: PATH + env
```

Code: fork-exec-wait-example1.c

fork-exec-wait-example1.c

```
pid_t pid = fork();
if (pid == 0) {                          // child
    printf("Child Process.\n");
    execlp("/bin/ls", "ls", (char *)0); // becomes ls
    printf("execlp() Failed");           // error path
    exit(-1);                            // only
} else {                                 // parent
    wait(NULL);
    printf("Parent Process.\n");
}
```

```
$ ./fork-exec-wait-example1
Child Process.
HelloWorld.c  fork-example1.c  makefile ...
Parent Process.
```



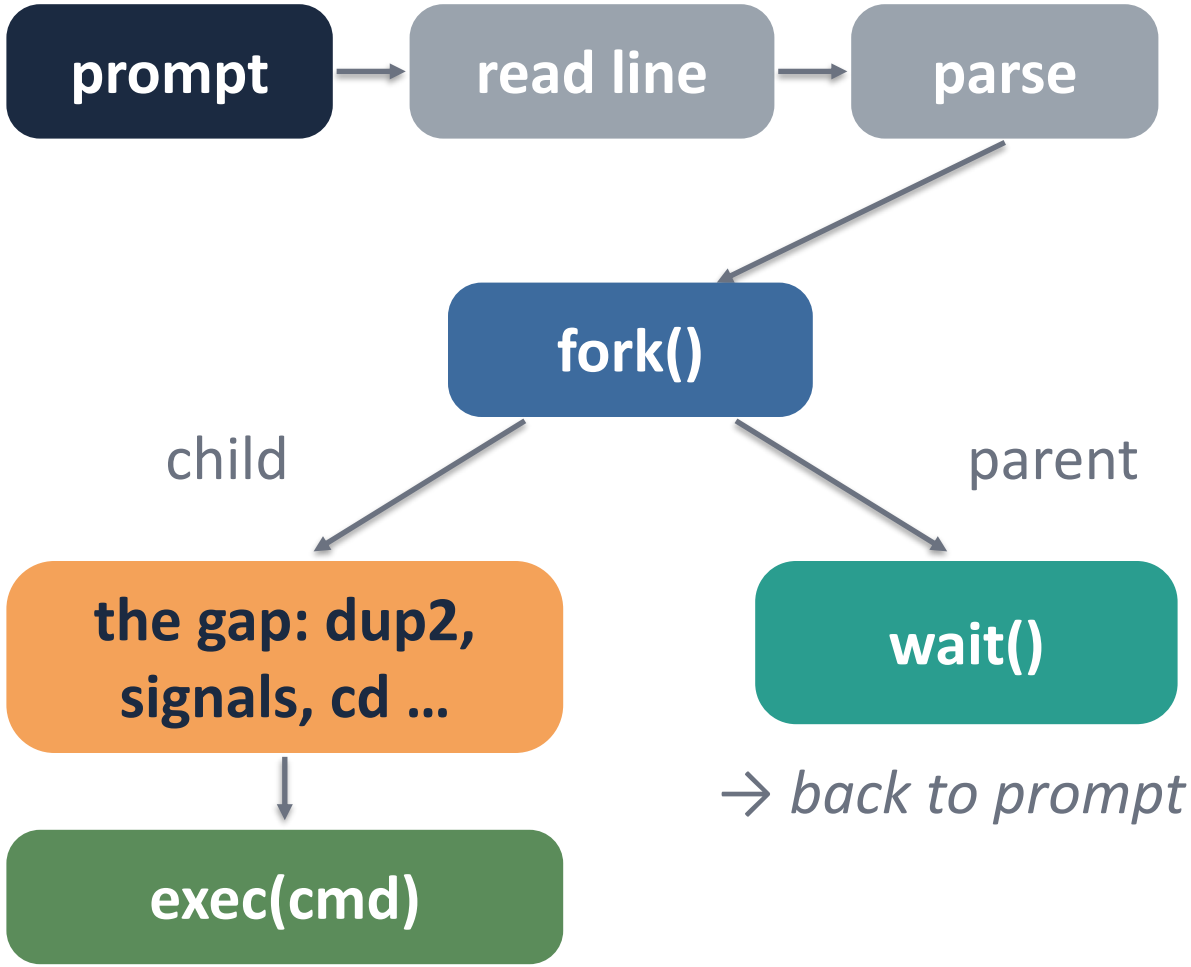
Worked example Q3

```
if (pid == 0) {
    printf("X");
    execvp("ls", args);
}
printf("Y");
```

→ 1 × X,
1 × Y

The child's Y never runs: after exec, its code is gone.

Why fork() and exec() are separate: the shell



- A shell is just a user program
- **The gap:** set up the child's world — redirect I/O, pipes, signals — without changing the program
- Windows CreateProcess() = both in one call
- Critique: "A fork() in the road" → posix_spawn()

Using the gap: I/O redirection

p4-redirect.c (OSTEP p4.c)

```
if (rc == 0) {                          // child
    close(STDOUT_FILENO);               // fd 1 free
    open("./p4.output",                 // gets fd 1 !
         O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
    char *args[] = {"wc", "p4.c", NULL};
    execvp(args[0], args);              // wc -> fd 1
} else {
    wait(NULL);
}
```

```
$ ./p4-redirect        # prints nothing
$ cat p4.output
   32   109   846 p4.c
```

start		close(1)		open()	
0	tty	0	tty	0	tty
1	tty	1	—	1	file
2	tty	2	tty	2	tty

then exec wc — same fd table

- open() returns the lowest free descriptor
- Open fds survive exec()
- wc never knows its stdout is a file

Next → the last piece — communication: signals and pipes

PART 5

Inter-process communication: signals and pipes

Where we are

- exec() replaces text/data/heap/stack; keeps PID and fds
- Seven variants: l / v / p / e
- The shell: fork → [gap] → exec, parent waits
- The gap enables redirection — and pipes

Questions for this part

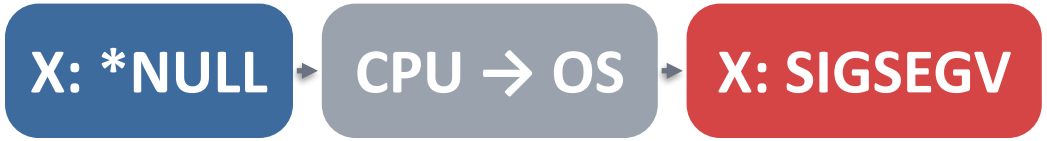
- Signals: a doorbell — one integer, asynchronous
- Pipes: a hose — a one-way byte stream
- Who may signal whom? users and the superuser (root)
- How does a shell build ls | grep txt ?

Signals: software interrupts

#	name	when
2	SIGINT	Ctrl-C
9	SIGKILL	cannot be caught
11	SIGSEGV	bad memory
13	SIGPIPE	write, no reader
17	SIGCHLD	a child ended
19	SIGSTOP	stop
18	SIGCONT	continue
20	SIGTSTP	Ctrl-Z

One integer, delivered by the kernel. kill -l lists them.

Synchronous: its own instruction



Asynchronous: from outside, any time



Also: timers (SIGALRM), the terminal (Ctrl-C), the kernel (SIGCHLD).

What a process can do with a signal

Default	Catch	Ignore	Mask
terminate, core, ignore, stop, cont	run a handler: signal(), sigaction()	SIG_IGN: “don't bother me”	sigprocmask(): hold it, pending

- `SIGKILL` and `SIGSTOP` cannot be caught, blocked, or ignored
- **fork():** the child inherits all dispositions
- **exec():** caught handlers → default; SIG_IGN stays ignored
- The PCB holds the handler vector, indexed by signal number

Assignment: if the shell ignores SIGINT, the child must restore SIG_DFL before exec

The signal API

call	purpose
`kill(pid, sig)`	send to a process
`raise(sig)`	send to myself
`signal(sig, h)`	install a handler
`sigaction(...)`	portable, richer
`pause()`	sleep until a signal
`alarm(sec)`	SIGALRM later

Handlers: keep them tiny; async-signal-safe calls only (write, not printf).

sigaction-usr1.c

```c
void h(int sig, siginfo_t *info, void *uc) {
    // demo only: printf isn't signal-safe
    printf("got %d from %d\n", sig,
           info->si_pid);
}
struct sigaction act = {0};
act.sa_sigaction = h;
sigemptyset(&act.sa_mask);
act.sa_flags = SA_SIGINFO; // sender info
sigaction(SIGUSR1, &act, NULL);
while (1) pause();
```

Try: kill -USR1 <pid> from another terminal.

Code: fork-signal.c — remote control over a child

fork-signal.c

```
void sighup(int s)  { printf("CHILD: got SIGHUP\n");
                      signal(SIGHUP, SIG_DFL); }
void sigint(int s)  { printf("CHILD: got SIGINT\n");
                      signal(SIGINT, SIG_DFL); }
void sigquit(int s) { printf("My DADDY killed me!\n");
                      exit(0); }
if (pid == 0) {                          // child
    signal(SIGHUP,  sighup);
    signal(SIGINT,  sigint);
    signal(SIGQUIT, sigquit);
    while (1) pause();
} else {                                 // parent
    kill(pid, SIGHUP);  sleep(3);
    kill(pid, SIGINT);  sleep(3);
    kill(pid, SIGQUIT); sleep(3);
}
```

```
$ ./fork-signal
PARENT: sending SIGHUP
CHILD: got SIGHUP
PARENT: sending SIGINT
CHILD: got SIGINT
PARENT: sending SIGQUIT
My DADDY killed me!
This is the END of program.
```

- SIG_DFL inside a handler resets the default
- pause() sleeps until a signal arrives
- exit(0) in the last handler: normal end

Signals a shell cares about: SIGCHLD and SIGINT

SIGCHLD — reap without blocking

```
void on_chld(int s) {
    int st;
    while (waitpid(-1, &st, WNOHANG) > 0)
        ; /* record st */
}
```

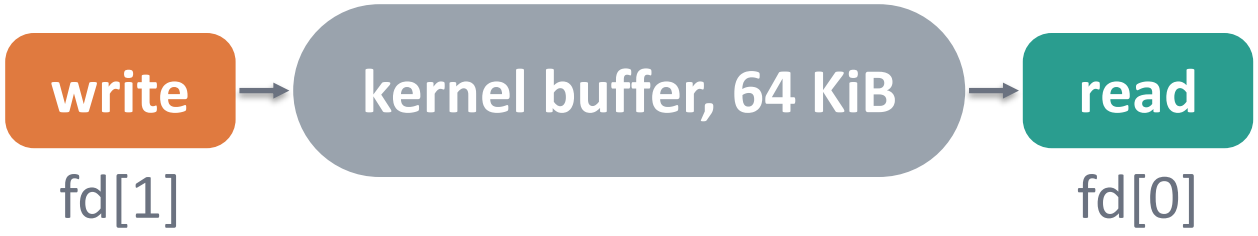
- Loop: signals do not queue — several children may have exited
- Or: foreground waitpid() after each command (the Assignment does this)
- Either way: no zombies

SIGINT — Ctrl-C hits the whole foreground group



- **Shell:** catch it → print a new prompt
- **Child:** restore SIG_DFL before exec
- **After waitpid:** report strsignal(WTERMSIG(st))

Pipes: pipe(), read(), write() — and fork()




```
int pipe(int fd[2]);   // fd[0] read, fd[1] write
ssize_t read (int fd, void *buf, size_t n);
ssize_t write(int fd, const void *buf, size_t n);
```

- pipe() before fork(): the child inherits both ends
- Each side closes the end it does not use
- read() returns 0 (EOF) only when all write ends are closed

fork-pipe.c

```
int fd[2];  char buf[100];
pipe(fd);
pid_t pid = fork();
if (pid == 0) {                 // child: reader
    close(fd[1]);               // unused end
    int n = read(fd[0], buf, 99);
    buf[n] = 0;
    printf("Child got: %s\n", buf);
    close(fd[0]);
} else {                        // parent: writer
    close(fd[0]);               // unused end
    write(fd[1], "Hello from parent", 17);
    close(fd[1]);               // EOF for reader
    wait(NULL);
}
```

Output: Child got: Hello from parent

Pipe rules: blocking, EOF, full buffer, SIGPIPE

situation	what happens	demo
read(), pipe empty, a writer open	blocks	`fork-pipe.c`
read(), pipe empty, no writers	returns 0 = EOF	`lab2-pipeerr.c`
write(), buffer full	blocks until drained	`lab2-pipefull.c`
write(), no reader	SIGPIPE (or –1, EPIPE)	`lab2-pipesig.c`

Close every unused end in every process — EOF and SIGPIPE depend on it.

The classic bug: a forgotten write end → the reader never sees EOF → the pipeline hangs.

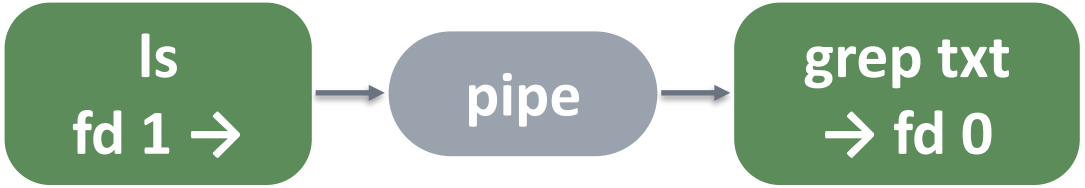
dup2() and building ls | grep txt

pipe-dup-fork.c

```
int fd[2];  pipe(fd);
if (fork() == 0) {                  // child 1: ls
    dup2(fd[1], STDOUT_FILENO);     // stdout -> pipe
    close(fd[0]); close(fd[1]);
    execlp("ls", "ls", NULL);
}
if (fork() == 0) {                  // child 2: grep
    dup2(fd[0], STDIN_FILENO);      // stdin <- pipe
    close(fd[0]); close(fd[1]);
    execlp("grep", "grep", "txt", NULL);
}
close(fd[0]); close(fd[1]);         // parent: no ends
wait(NULL); wait(NULL);
```

6 calls you already know, composed.

```
int dup2(int oldfd, int newfd); // newfd=oldfd
```



N commands (pipeline-n.c):

- N−1 pipes, created before forking
- child i: dup2 prev read → 0; dup2 pipe[i] write → 1
- close ALL pipe fds, then exec
- parent: close all, then waitpid × N

SUMMARY

The life of a process — in one line



signals: kill() → child · pipes: child ⇄ child · /proc: the PCB as text

- A process = running program = address space + registers + resources; tracked by a PCB, moved between states by the scheduler
- fork() clones (copy-on-write) and returns twice; order is up to the scheduler
- exit/signal → zombie until wait*() reaps it; decode with WIFEXITED / WIFSIGNALED
- exec() swaps the program, keeps PID and fds — fork–[gap]–exec–wait is the shell
- Signals = control (dispositions inherited by fork, reset by exec); pipes = data (close every unused end)

Cheat sheet: headers, calls, man pages

header	calls
<unistd.h>	`fork getpid getppid exec* pipe dup2 read write close pause`
<sys/wait.h>	`wait waitpid waitid wait4, WIF*/WEXITSTATUS/WTERMSIG, WNOHANG WNOWAIT`
<signal.h>	`kill raise signal sigaction sigprocmask sigsuspend`
<sys/resource.h>	`struct rusage`
<stdlib.h> <string.h>	`exit strsignal perror strerror errno`

RTFM: `man 2 fork` · `man 2 waitid` · `man 7 signal` · `man 5 proc`

Summary

ASSIGNMENT 1

Yet Another Shell — 3230yash

- Due: See in the moodle · 15 points
- Assessment for ILO4 [Practicability]; learning activity for ILO 2a
- Goal: design and develop an interactive shell in C — create, manage and coordinate multiple processes
- Practise: process creation & program execution, signals and pipes, obtaining a process's runtime statistics

What 3230yash must support

- Prompt `## 3230yash >>` ; a line ≤ 1024 chars, ≤ 30 strings (incl. |)
- Run any program with its arguments in a new process; wait for it before the next prompt
- Locate programs by absolute path, relative path (./), or the $PATH directories
- **Two built-ins:** `exit` and `watch`
- Chain commands with | (up to 4 pipes = 5 commands)
- 3230yash itself must not be terminated by Ctrl-C (SIGINT)

`a session (from the spec)`

```
## 3230yash >> ls -la
...
## 3230yash >> cat TP.html | grep table | wc -l
6
## 3230yash >> watch ./add
STATE CPUID UTIME STIME VSIZE   MINFLT MAJFLT
R     18    0.00  0.00  2748416 10     0
R     18    0.49  0.00  2609152 61     2
Z     18    1.27  0.00  0       65     2
## 3230yash >> ./loopf 10
^Cloopf: Interrupt
## 3230yash >> ^C
## 3230yash >> exit
3230yash: Terminated
```

Interactive use; the full details are in the Specification section.

Spec 1: running commands — 4.5 points

- Run the command, with its arguments, in a new process
- `/home/tmchan/a.out` · `./a.out` · `gcc hello.c` via $PATH
- Cannot locate or execute → error message, prompt again
- Wait for the child before the next prompt
- Child killed by a signal → print its name and the signal
- **Graded:** prompt, new process, path lookup, arguments, errors, waiting, signal report, repeated commands, **no zombies**

`expected messages`

```
## 3230yash >> simp
3230yash: 'simp': No such file or directory
## 3230yash >> /bin/simp
3230yash: '/bin/simp': No such file or directory
## 3230yash >> ./start.sh
3230yash: './start.sh': Permission denied
## 3230yash >> ./loopf 10
^Cloopf: Interrupt
## 3230yash >>
```

Messages must match the specification exactly.

Spec 2: pipelines with | — 4 points

- Each | connects the stdout of one command to the stdin of the next
- Up to 4 pipes (5 commands); each command may have any arguments
- Invalid: | as the first or last symbol → "Incorrect pipe sequence"
- Invalid: two consecutive | → "should not have two consecutive | without in-between command"
- **Graded:** correct use & error reporting · two commands · two commands with arguments · up to five commands with arguments

valid and invalid input

```
## 3230yash >> cat TP.html | grep table | wc -l
6
## 3230yash >> | wc -l
3230yash: Incorrect pipe sequence
## 3230yash >> cat abc.txt |
3230yash: Incorrect pipe sequence
## 3230yash >> cat hello.c | | wc -l
3230yash: should not have two
  consecutive | without in-between command
```



stdout → stdin

Spec 3: built-ins exit and watch

exit — 0.5 pt

- `exit` alone → release resources, print “3230yash: Terminated”, quit
- `exit now` → “"exit" with other arguments!!!”, keep running
- `not exit` → an ordinary (unknown) command

watch <cmd> — 4 pts

- Monitors one child until it terminates, then reports
- Snapshots: before it starts · every 500 ms · after it terminates
- Columns: STATE CPUID UTIME STIME VSIZE MINFLT MAJFLT
- Data from /proc/{pid}/stat; non-blocking waitid()
- Pipelines not allowed → “Cannot watch a pipe sequence”
- **Graded:** usage & errors · before / during / after · shown after termination

Spec 4: SIGINT handling — 2 points

- 3230yash must not be terminated by SIGINT (Ctrl-C or kill)
- Ctrl-C while waiting for input → simply a new prompt
- Children respond per their own program — some terminate, some don't
- Child terminated by a signal → “<name>: <signal>”, e.g. “Interrupt”, “Killed”
- **Graded:** 3230yash survives SIGINT · children may be terminated by it

expected behaviour

```
## 3230yash >> ./forever
^CReceives SIGINT!! IGNORE IT :)
^CReceives SIGINT!! IGNORE IT :)
forever: Killed
## 3230yash >> ./loopf 10
^Cloopf: Interrupt
## 3230yash >> ^C
## 3230yash >> ^C
## 3230yash >>
```

See today's Part 5 for the background on signals.

Grading, documentation, platform, submission

component	pts
Process creation & execution	4.5
Pipelines (|)	4
SIGINT signals	2
Built-in watch	4
Built-in exit	0.5
Missing documentation	−1

Total: 15 points — correctness of the program

- **Documentation:** name & number, development platform, how much is completed; detailed inline comments
- **Platform:** C, compiled with gcc; must run on workbench2 (via HKUVPN) — the grading machine; no compile = 0
- **Submit:** 3230yash_{UID}.c, zipped (zip/tgz), on Moodle
- **GenAI:** allowed with a report — models, prompts & responses, adopted snippets
- **Plagiarism** is a serious offence; you may be asked to explain your program

Where today's tutorial meets the Assignment

requirement	background	slides
run a command; wait; report signals	Parts 2, 3, 4	12–13, 20–23, 27–29
locate programs (path, ./, $PATH)	exec family	28
pipelines with |	Part 5: pipes	38–40
SIGINT handling	Part 5: signals	33–37
watch: /proc stats, waitid	Part 3	24–25
no zombies	Part 3	19–22

Read the specification carefully — the graders type exactly the cases it describes.

Thank you — questions by email.